Horizon Adaptation In RL-Assisted MPC

2026-04-01

Method Goal

This method learns when the offset-free MPC controller should use different prediction and control horizons. The key idea is that the actor does not change continuous inputs directly. Instead, a DQN policy selects one discrete horizon recipe from a finite menu and the MPC is rebuilt around that choice.

Notebook Sources Covered

- `RL_assisted_MPC_horizons.ipynb`
- `RL_assisted_MPC_horizons_dist.ipynb`
- Shared helpers from `utils/helpers.py`, `DQN/dqn_agent.py`, and `Simulation/mpc.py`

Shared MPC Context

All horizon experiments reuse the same baseline ingredients:

- a nonlinear polymer CSTR plant for closed-loop rollout,
- a linear augmented model $(A_{\text{aug}}, B_{\text{aug}}, C_{\text{aug}})$ for offset-free MPC,
- a scaled observer state $\hat{x}_t$,
- scaled setpoint deviations and scaled input deviations.

The MPC objective implemented in `Simulation/mpc.py` is

$$J = \sum_{k=1}^{H_p} \|y_{t+k} - y_{\text{sp}}\|_{Q_{\text{out}}}^2 + \sum_{k=1}^{H_c} \|\Delta u_{t+k-1}\|_{R_{\text{in}}}^2.$$

The horizon agent changes H_p and H_c while leaving the optimizer form itself unchanged.

State, Action, Reward, And Update Logic

The RL state is built from the scaled observer state, scaled setpoint deviation, and scaled current input deviation:

$$s_t = \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_{\mathrm{sp},t} \\ \tilde{u}_t^{\mathrm{dev}} \end{bmatrix}.$$

The action is discrete:

$$a_t \in \{0, 1, \ldots, |\mathcal{H}| - 1\}, \quad \mathcal{H} = \{(H_p, H_c) \mid H_c \leq H_p\}.$$

The selected index is converted to a horizon pair through `action_to_horizons`. The reward is still computed from tracking and input-move quality after the MPC solve. In the notebook family, the shaped reward is multiplied by 0.01 before it is stored in the DQN replay buffer.

The DQN update follows a Double-DQN target:

$$y_t = r_t + \gamma Q_{\theta^-}\left(s_{t+1}, \arg\max_a Q_\theta(s_{t+1}, a)\right).$$

Core Algorithm Flow

1. Build a finite recipe table of valid (H_p, H_c) pairs.
2. Form the RL state from the scaled observer state and current operating point.
3. Let the DQN choose a horizon index, usually only every decision interval.
4. Rebuild the MPC object if the selected horizon pair changed.
5. Solve the MPC optimization with the active H_p, H_c, apply the first move to the plant, and update the observer.
6. Store (s_t, a_t, r_t, s_{t+1}) in the DQN replay buffer and run `train_step()` after warm start.

Key Code Paths

Reusable helper for horizon recipes

```
def build_horizon_recipes(predict_grid, control_grid):
    recipes = []
    for Hp in predict_grid:
        for Hc in control_grid:
            if Hc <= Hp:
                recipes.append((Hp, Hc))
    if not recipes:
        raise ValueError("No valid Horizon recipes found")
    return recipes

def action_to_horizons(horizon_recipes, a_idx):
    return horizon_recipes[a_idx]
```

Notebook-local supervisor logic

```
if i > WARM_START:
    if (i % DECISION_INTERVAL == 0) or (last_a is None):
        if not test:
            a_idx = agent.take_action(current_rl_state.astype(np.float32),
                                      eval_mode=test)
        else:
            a_idx = agent.act_eval(current_rl_state.astype(np.float32))
        Hp, Hc = action_to_horizons(h_recipes, a_idx)
        if (Hp, Hc) != (current_Hp, current_Hc):
            rebuild_mpc(Hp, Hc)
            current_Hp, current_Hc = Hp, Hc
        last_a = a_idx
else:
    a_idx = a_idx_mpc
    Hp, Hc = action_to_horizons(h_recipes, a_idx)
```

DQN action selection and training

```
@torch.no_grad()
def take_action(self, state, eval_mode=False):
    self.steps += 1
    eps = 0.0 if eval_mode else self.eps_schedule.value(self.steps)
    if random.random() < eps:
        return random.randrange(self.action_dim)
    s = torch.from_numpy(state).float().unsqueeze(0).to(self.device)
    q1 = self.online.q1_forward(s)
    return int(q1.argmax(dim=1).item())

def train_step(self):
    ...
    a_star = self.online.q1_forward(ns).argmax(dim=1)
    q_t = self.target.combined_forward(ns, mode=self.target_combine)
    q_next = q_t.gather(1, a_star.view(-1, 1)).squeeze(1)
    y = r + self.gamma * q_next * (1 - done)
```

Variants Covered

- The nominal notebook studies horizon adaptation without injected operating-condition drift.
- The disturbance notebook adds gradual changes to process quantities such as Q_i, Q_s, and hA through the shared schedule generator.
- Both variants keep the same DQN-based horizon abstraction and the same observer-based state pipeline.

Limitations, Caveats, And Open Questions

- The true execution logic is notebook-local. The core supervisor is not fully lifted into a reusable module.
- The horizon decision interval is controlled by notebook globals rather than a shared config layer.
- Reward scaling by 0.01 is specific to this notebook family and should be preserved if results are compared historically.
- `Simulation/rl_sim.py` is not the source of truth for this method; the notebook-local supervisor is.